

Enatega Admin Panel — Security, Performance & Code-Cleanup Audit

Module: enatega-multivendor-admin **Branch:** adeel **App type:** Admin dashboard (super-admin + vendor/restaurant owner) **Stack:** Next.js 14.2.5 (App Router) · Apollo Client 3.13 · GraphQL · PrimeReact · TailwindCSS · Firebase (FCM) · next-intl **Audit date:** 2026-06-29 **Auditor scope:** Frontend code only. The GraphQL backend/API is proprietary and **not present in this repository** — every backend-dependent claim is explicitly marked as such.

Executive Summary

The Enatega Admin panel is a feature-complete Next.js App-Router dashboard with clean role-based route grouping and broadly safe rendering (no `dangerouslySetInnerHTML`, no `eval`, Formik+Yup validation, confirmation dialogs on destructive actions). However, the **session/authorization model is the weakest area**: authentication state, role (`userType`) and permissions are stored in `localStorage` and trusted client-side; route guards run in a `useEffect` with empty dependencies and never gate rendering, so protected screens render and fire their queries before any redirect; and **logout never clears the bearer token** (`clearAuthTokens/clearMetricsData` are defined but never called), leaving a valid token in `localStorage` after sign-out. The GraphQL configuration query pulls **server-side secrets** (Stripe `secretKey`, `clientSecret`, SMTP password, `twilioAuthToken`, `sendGridApiKey/sendGridPassword`) into the browser. Dependencies carry **30 known vulnerabilities (3 critical / 14 high)** including the Next.js cache-poisoning advisories and the `lodash_.template` code-injection issue. Performance is dominated by a **fetch-everything-then-filter-client-side** pattern (`fetchPolicy: 'network-only'` used 55×, full tables loaded into memory and paginated with `Array.slice`) plus a 654-line dummy-data generator shipped to production as loading skeletons. Cleanup opportunities include 25 orphaned locale files, legacy white-label branding URLs (`hedgego.com.au`, `ninjascode.com`), an unused modern WS client dependency, and the default Cypress scaffold standing in for a real test suite.

Overall risk rating: HIGH (driven by session handling, client-trusted authorization, secret exposure in the config query, and the dependency CVE surface).

Scope

In scope	Out of scope
All source under enatega-multivendor-admin/ (app/, lib/, i18n/, public/, root config)	Backend/GraphQL resolver code (proprietary, not in repo)
Auth/session/authorization handling as implemented on the client	Actual server-side authorization enforcement
GraphQL client setup, queries/mutations as	Runtime penetration testing / live exploitation

In scope	Out of scope
defined client-side	
Dependency manifest + <code>npm audit</code>	Other monorepo modules (<code>-web</code> , <code>-app</code> , <code>-rider</code> , <code>-store</code>)
Bundle/render/state/build performance patterns	Infra / CDN / WAF configuration
Dead, duplicate, and legacy code	

Methodology

1. **Structure mapping** — enumerated `app/` route groups, `lib/` subtrees, GraphQL operation files, contexts, hooks, and config files.
 2. **Static reading** — read the security-critical files end-to-end (Apollo setup, auth/security helpers, route guards, user/configuration contexts, login + logout flows).
 3. **Pattern sweeps** — `grep` for XSS sinks (`dangerouslySetInnerHTML`, `innerHTML`, `eval`), redirects (`window.open`, `location.href`), secrets, `console.*`, `fetchPolicy`, `pollInterval`, `React.memo/useMemo/useCallback`, library imports, and legacy branding.
 4. **Dependency analysis** — `npm audit` + installed-version checks + per-package usage greps to separate *used* from *unused* and *direct* from *transitive*.
 5. **Evidence binding** — every finding below cites a real `path:line` with the quoted code. Where a conclusion depends on backend behavior that is not in this repo, it is labeled **Cannot be verified from the current codebase**.
-

Project Structure Map

Framework & tooling

- **Next.js 14.2.5** App Router (`next.config.mjs`, wrapped with `next-intl/plugin`).
- **React 18**, TypeScript `strict: true` (`tsconfig.json`), path alias `@/*`.
- **Apollo Client 3.13.0** (manifest pins `^3.11.8`).
- **PrimeReact 10.8.3** + Tailwind 3.4.1 for UI.
- **Node ≥ 20** enforced (`.nvmrc v20.16.0`, `.npmrc engine-strict=true`).
- **Cypress 13** present but only the default scaffold test exists.
- **742** `.ts/.tsx` files under `lib/` + `app/`.

Routing (App Router, route groups gate role)

```
app/
  layout.tsx # root
  (localized)/
    layout.tsx # next-intl locale wrapper
    global.css
    forbidden/page.tsx # 403 landing
    (unprotected)/authentication/ # login, sign-up, otp, verify-
email, verify-phone
    (protected)/
      layout.tsx # GoogleMaps + GlobalLayout (NO
auth check here)
      page.tsx
      (super-admin)/
        layout.tsx # super-admin shell (Firebase
init, sidebar)
      home/ , general/{users,riders,staff,stores,vendors}/ ,
      management/{banners,commission-rates,configurations,coupons,
      cuisines,notifications,orders,shop-types,tippings}/ ,
      wallet/{earnings,transaction-history,withdraw-requests}/ ,
      audit-logs/ , customerSupport/ ,
      (app-bar-routes)/{dispatch,language,settings,zone}/
      (other-users)/
        layout.tsx
        admin/store/ ... (coupons, dashboard, orders, product-management,
        profile, ratings, wallets,
general/{location,payment,timing})
        admin/vendor/ ... (dashboard, profile, stores)
```

Role gating is applied **per screen** through HOCs in `lib/hoc/` (`SUPER_ADMIN_GUARD.tsx`, `VENDOR_GUARD.tsx`, `RESTAURANT_GUARD.tsx`) — **not** in `(protected)/layout.tsx`, which performs no auth check.

lib/ structure

Path	Purpose
<code>lib/api/graphql/</code>	<code>queries/</code> (28 files), <code>mutations/</code> (28 files), <code>subscription/</code> (3 files), <code>index.ts</code> barrel
<code>lib/context/</code>	<code>global/</code> (configuration, user, layout, sidebar, toast, google-maps — 15 context files total), <code>super-admin/</code> , <code>vendor/</code> , <code>restaurant/</code>
<code>lib/hooks/</code>	<code>useSetApollo</code> , <code>useQueryQL</code> , <code>useLazyQueryQL</code> , <code>useUser</code> , <code>useConfiguration</code> , ... (10 files)
<code>lib/hoc/</code>	the three route guards
<code>lib/services/</code>	<code>cloudinary.ts</code> , <code>index.ts</code>
<code>lib/ui/layouts/</code>	<code>protected/unprotected</code> shells (super-admin, vendor, restaurant, home, global)
<code>lib/ui/screen-components/</code>	feature screens (super-admin, vendor, restaurant, unprotected/auth)
<code>lib/ui/useable-components/</code>	~60 shared primitives (table, dialogs, inputs,

Path	Purpose
	google-maps, charts)
lib/utils/	interfaces/ (66), schema/ (25 Yup), constants/, methods/ (auth, security, local-storage, error, ...), dummy/ (654-line mock generator)

Key cross-cutting files

Concern	File
Apollo client + auth header injection + WS	lib/hooks/useSetApollo.tsx
Auth token storage helpers	lib/utils/methods/auth.ts
"Metrics"/bop-auth public token (obfuscated)	lib/utils/methods/security.ts
localStorage wrapper	lib/utils/methods/local-storage.ts
User session state	lib/context/global/user-context.tsx
Server-driven runtime config (incl. secrets)	lib/context/global/configuration.context.tsx + lib/api/graphql/queries/configuration/index.ts
Login	lib/ui/screen-components/unprotected/authentication/sign-in-email-password/index.tsx
Logout	lib/ui/screen-components/protected/layout/{super-admin, vendor, restaurant}-layout/app-bar/index.tsx
Role permission list	lib/utils/constants/permissions.ts
Firebase FCM init	firebase.ts, public/firebase-messaging-sw.js
Image / build config	next.config.mjs
i18n	config.ts, i18n/request.ts, locales/*.json

Domain screens present

- **Order management:** super-admin management/orders, vendor order, store orders, dispatch.
- **Vendor/Restaurant/Rider/Customer management:** general/{vendors, stores, riders, users, staff}, vendor & restaurant

scoped screens.

- **Payment-related (admin config):** management/configurations add-forms for **Stripe, PayPal, Twilio, Nodemailer/SendGrid, Cloudinary, Firebase-admin, Google** keys; restaurant general/payment.
- **File/image handling:** lib/services/cloudinary.ts, browser-image-compression, lib/utils/methods/image, lib/utils/methods/video.
- **Notifications/Firebase:** firebase.ts, public/firebase-messaging-sw.js, management/notifications.
- **Wallet/earnings/withdrawals, audit-logs, customer support chat** (Apollo subscriptions).

Test setup

Only the **default Cypress scaffold** exists:

cypress/e2e/1-getting-started/todo.cy.ts,

cypress/fixtures/example.json, cypress/support/{commands, e2e}.ts. No application-specific E2E or unit tests.

Findings Count Table

Severity	Security	Performance	Cleanup	Total
Critical	1	0	0	1
High	4	2	0	6
Medium	6	3	3	12
Low	3	2	4	9
Info	1	0	1	2
Total	15	7	8	30

Top Security Risks

Priority	Issue	Impact	Location	Effort
1	Logout does not clear the bearer token	Token reuse / session not terminated	.../super-admin-layout/app-bar/index.tsx:163	Low
2	Server-side secrets fetched into the browser	Stripe/SMTP/Twilio/SendGrid secret exposure	lib/api/graphql/queries/configuratio	Med (backend)

Priority	Issue	Impact	Location	Effort
			n/index.ts	
3	Authorization trusts userType/permissions from localStorage	Privilege escalation if backend not enforcing	lib/hoc/ *_GUARD.tsx, user- context.tsx	Med
4	Guards don't gate render (empty- deps useEffect, no early return)	Protected screens render + query before redirect	lib/hoc/ SUPER_ADMIN_ GUARD.tsx:21- 51	Low
5	Tokens in localStorage	XSS → full token/PII exfiltration	lib/utils/ methods/ auth.ts:15- 19	Med
6	30 dependency CVEs (Next.js, lodash, next- intl...)	RCE/cache- poisoning/ReDoS exposure	package.json	Med
7	Apollo connectToDev Tools: true in all environments	Schema/ operation/data introspection in prod	lib/hooks/ useSetApollo .tsx:176	Low

Top Performance Wins

Priority	Issue	Estimated impact	Location	Effort
1	Fetch-all-rows + client-side filter/paginate	O(all rows) memory & transfer; slow large tables	.../users/ view/main/ index.tsx:39- 82 (+ pattern across view/main)	Med
2	fetchPolicy: 'network- only' used 55×	Cache bypass; refetch on every mount/nav	repo-wide	Med
3	654-line dummy- data generator shipped to prod	Larger bundle; fake data on every screen	lib/utils/ dummy/ index.tsx (24 importers)	Med
4	moment bundled for 1 usage; date-fns	~Heavy date lib shipped needlessly	.../audit- logs/ AuditLogCard .tsx	Low

Priority	Issue	Estimated impact	Location	Effort
	unused			
5	<code>images.unoptimized: true</code> (global)	No Next image optimization; large payloads	<code>next.config.mjs:7</code>	Low
6	Live-monitor polls every 15s with network-only	Steady background query load	<code>.../dashboard/live-monitor/index.tsx:36</code>	Low

Top Cleanup Opportunities

Priority	Code to remove/refactor	Why	Location	Removal risk
1	Wire up (don't delete) <code>clearAuthTokens/clearMetricsData</code>	Defined but never called; logout gap	<code>lib/utils/methods/auth.ts:32, security.ts:84</code>	Medium
2	25 orphaned locale JSON files	Only 6 locales declared in <code>config.ts</code>	<code>locales/*.json</code>	Medium
3	Legacy white-label branding URLs	<code>hedgego.com.au</code> , <code>ninjascode.com</code> not Enatega	<code>side-bar-item.tsx:97-98, super-admin/index.tsx:106</code>	Medium
4	Default Cypress scaffold	Placeholder, not a real test	<code>cypress/e2e/1-getting-started/todo.cy.ts</code>	Safe
5	<code>graphql-ws</code> dependency	Declared but unused (uses <code>subscriptions-transport-ws</code>)	<code>package.json</code>	Safe
6	<code>console.*</code> debug statements (88)	Noise + PII/key leakage	repo-wide	Safe

PART 2 — Security Findings

[CRITICAL] Server-side secrets are delivered to the admin browser via the configuration query

- **Category:** Security / Secrets / Sensitive Data Exposure
- **Location:** `lib/api/graphql/queries/configuration/index.ts`
(`GET_CONFIGURATION`) → consumed in
`lib/context/global/configuration.context.tsx`
- **Description:** The `getConfiguration` query requests fields that are secrets, not public config. They land in the Apollo `InMemoryCache`, in `ConfigurationContext`, and are rendered into the configuration add-forms
(`lib/ui/screen-components/protected/super-admin/configuration/add-form/{stripe,paypal,twilio,nodemailer}/...`).
- **Evidence:**

```
# lib/api/graphql/queries/configuration/index.ts
configuration {
  email
  password          # SMTP password
  clientSecret       # PayPal client secret
  secretKey          # Stripe SECRET key
  twilioAccountSid
  twilioAuthToken    # Twilio auth token
  sendGridApiKey
  sendGridPassword
  ...
}
```

- **Impact:** Stripe secret key, PayPal client secret, Twilio auth token, SendGrid API key and SMTP password are transmitted to every authenticated admin browser. With them an attacker can move money / send messages / send mail as the platform. Whether the backend should ever return these is a server design issue, but the **client explicitly asks for them**, so they are exposed regardless.
- **Reproduction / How to verify:** Log in as any admin → DevTools → Network → the `getConfiguration` GraphQL response; or Apollo DevTools cache → `Configuration` object. Observe `secretKey`, `clientSecret`, `twilioAuthToken`, `sendGridPassword`, `password` populated.
- **Recommended fix:** Remove secret fields from the client query. The backend should expose only publishable values (`publishableKey`, `clientId`, `googleApiKey`, `firebaseKey`, etc.) and mask/omit secrets, or gate secret read behind a write-only "set, never read" pattern. Client config forms should treat secrets as write-only (show `••••`, never echo the stored value).
- **References:** OWASP A02:2021 Cryptographic Failures; OWASP API3:2023 Excessive Data Exposure; CWE-200, CWE-312.
- **Backend dependency:** The fix ultimately requires backend cooperation. **Cannot be**

verified from the current codebase whether the server filters these by role — but the client unconditionally requests them.

[HIGH] Logout does not clear the auth token (session not terminated client-side)

- **Category:** Security / Session
- **Location:** lib/ui/screen-components/protected/layout/super-admin-layout/app-bar/index.tsx:163-170 (and the vendor/restaurant app-bars: vendor-layout/app-bar/index.tsx:122, restaurant-layout/app-bar/index.tsx:127)
- **Description:** onConfirmLogout removes user-`${APP_NAME}` and the `SELECTED_*` keys but **never calls `clearAuthTokens()` or `clearMetricsData()`**. The token, userType, userId, tokenExpiration keys written by `setAuthTokens` (and the obfuscated metrics token) persist. Because the Apollo request middleware prefers `getAccessToken()` (the raw token key), requests made after "logout" still carry the old Bearer token until it is read back at next login.
- **Evidence:**

```
// app-bar/index.tsx:163
const onConfirmLogout = () => {
  setUser(null);
  onUseLocalStorage('delete', SELECTED_VENDOR);
  onUseLocalStorage('delete', SELECTED_VENDOR_EMAIL);
  onUseLocalStorage('delete', SELECTED_RESTAURANT);
  onUseLocalStorage('delete', `user-${APP_NAME}`); // <--
  'token', 'userType', 'userId', 'tokenExpiration' NOT removed
  router.push('/authentication/login');
};

// lib/utils/methods/auth.ts:32 - defined but never called anywhere
export function clearAuthTokens(): void {
  if (typeof window === 'undefined') return;
  Object.values(AUTH_KEYS).forEach((key) => localStorage.removeItem(key));
}

// lib/hooks/useSetApollo.tsx:114 - request still uses getAccessToken() first
const token = getAccessToken() || (data ? JSON.parse(data).token : '');
```

- **Impact:** On a shared/kiosk machine, the next user can read the previous admin's token from `localStorage` and replay it until expiry. Session termination is incomplete.
- **Reproduction / How to verify:** Log in → DevTools → Application → Local Storage: confirm token, userType, userId, tokenExpiration present. Click Logout → re-inspect: those keys remain. `grep -rn "clearAuthTokens\|clearMetricsData"` shows only the definitions, no call sites.
- **Recommended fix:**

```
import { clearAuthTokens } from '@lib/utils/methods/auth';
import { clearMetricsData } from '@lib/utils/methods/security';
```

```
const onConfirmLogout = async () => {
  setUser(null);
  clearAuthTokens();
  clearMetricsData();
  ['SELECTED_VENDOR', 'SELECTED_VENDOR_EMAIL', 'SELECTED_RESTAURANT', `user-${APP_NAME}`, 'messaging-token']
    .forEach(k => onUseLocalStorage('delete', k));
  await apolloClient.clearStore(); // drop cached config secrets / PII
  router.replace('/authentication/login');
};
```

- **References:** OWASP A07:2021 Identification & Authentication Failures; CWE-613 Insufficient Session Expiration.

[HIGH] Authorization decisions trust userType/permissions read from localStorage

- **Category:** Security / Authorization (AuthZ)
- **Location:** lib/context/global/user-context.tsx:17-24;
lib/hoc/SUPER_ADMIN_GUARD.tsx:32-48;
lib/hoc/VENDOR_GUARD.tsx:26-37;
lib/hoc/RESTAURANT_GUARD.tsx:28-34
- **Description:** The session user object — including userType and permissions — is rehydrated directly from the user-`\${APP\_NAME}` localStorage entry, then used to allow/deny screens. A user can edit that JSON in DevTools (e.g. set userType: "SUPER ADMIN" or add permission strings) to satisfy every client-side gate.
- **Evidence:**

```
// user-context.tsx:17
const savedUser = localStorage.getItem(`user-${APP_NAME}`);
if (savedUser) setUser(JSON.parse(savedUser)); // userType/permissions trusted as-is

// SUPER_ADMIN_GUARD.tsx:32
if (user && user.userType === 'STAFF' && findRouteName &&
Array.isArray(user.permissions)) {
  const allowed = user?.permissions?.includes(findRouteName?.text);
  if (!allowed) router.replace('/forbidden');
}
if (user?.userType === 'RESTAURANT' || user?.userType === 'VENDOR')
router.replace('/forbidden');
```

- **Impact:**
 - **Client-side risk (verified):** all UI gates can be bypassed by editing localStorage. The user reaches the screen and its queries fire.
 - **Backend-dependent risk:** whether the GraphQL API independently authorizes each operation by the JWT's real role. **Cannot be verified from the current codebase.** If the backend relies on the client gate, this is a full privilege-escalation / BOLA risk.

- **Reproduction / How to verify:** Log in as a vendor/staff → DevTools → Application → edit user-enatega... JSON → set "userType": "ADMIN" (or "SUPER ADMIN") and reload. Observe whether super-admin routes render and whether their mutations succeed (the latter proves/disproves backend enforcement).
 - **Recommended fix:** Keep client gating for UX only; ensure the **server** authorizes every query/mutation from the verified JWT claims (not request body / not client role). Do not store role in a mutable client store as the source of truth; derive it from a verified token if it must be client-visible.
 - **References:** OWASP A01:2021 Broken Access Control; OWASP API1/API5:2023 (BOLA/BFLA); CWE-602, CWE-639.
-

[HIGH] Route guards do not gate rendering (empty-deps effect + unconditional render)

- **Category:** Security / Authorization
- **Location:** lib/hoc/SUPER_ADMIN_GUARD.tsx:21-51 (same shape in VENDOR_GUARD.tsx, RESTAURANT_GUARD.tsx)
- **Description:** Each guard runs its login/role checks inside `useEffect(() => {...}, [])` (empty dependency array) and then **always returns** `<Component {...props} />`. Two problems: (1) on first render user is null (the `UserProvider` populates it in its own `useEffect`), so the role branch is skipped entirely; (2) there is no loading state or early `return null` before the redirect resolves, so the protected screen mounts, runs its GraphQL queries, and briefly paints before `router.replace` navigates away.
- **Evidence:**

```
// SUPER_ADMIN_GUARD.tsx:21
useEffect(() => {
  const isLoggedIn = !!onUseLocalStorage('get', `user-${APP_NAME}`);
  if (!isLoggedIn) router.replace('/authentication/login');
  ...role checks using `user`...
}, []); // empty deps: does not re-run when `user` loads
return <Component {...props} />; // renders regardless of the checks
```

- **Impact:** Unauthorized data is fetched (queries fire) and can flash on screen before redirect. Combined with the localStorage-trust finding above, the guard provides no real protection.
- **Reproduction / How to verify:** Throttle network (DevTools → Slow 3G), navigate directly to a super-admin URL while logged in as a vendor: the page content/queries appear momentarily before redirect to `/forbidden`.
- **Recommended fix:** Add `user` (and `pathname`) to the dependency array, render a loader until `user` resolves, and `return null` (or a spinner) while unauthorized:

```
const { user, loading } = useUserContext();
if (loading || user === null) return <FullPageLoader />;
if (!isAllowed(user, pathname)) { router.replace('/forbidden'); return null; }
```

```
return <Component {...props} />;
```

- **References:** OWASP A01:2021; CWE-285 Improper Authorization.
-

[HIGH] Auth token & full login payload stored in `localStorage`

- **Category:** Security / Auth / Sensitive Data Exposure
- **Location:** `lib/utils/methods/auth.ts:15-19`; `login.../sign-in-email-password/index.tsx:73-79`
- **Description:** The JWT is stored in plain `localStorage` (token key) and the **entire** `ownerLogin` response (token + identity) is additionally stored as JSON under `user-${APP_NAME}`. `localStorage` is readable by any JavaScript in the origin, so any XSS (including a malicious dependency or a future `dangerouslySetInnerHTML`) exfiltrates the session.
- **Evidence:**

```
// auth.ts:15
if (payload.token) localStorage.setItem(AUTH_KEYS.TOKEN, payload.token);

// sign-in-email-password/index.tsx:73
onUseLocalStorage('save', `user-${APP_NAME}`, JSON.stringify(ownerLogin));
setAuthTokens({ userId, token, tokenExpiration, userType });
```

- **Impact:** Token theft via XSS; no `HttpOnly/Secure/SameSite` protection possible for `localStorage`.
 - **Reproduction / How to verify:** DevTools → Application → Local Storage shows token and `user-enatega...` (with token) in cleartext.
 - **Recommended fix:** Prefer an `HttpOnly, Secure, SameSite=Strict` cookie set by the backend for the session token; if `localStorage` must stay, minimize stored fields (never store the full payload), shorten token lifetime, and add a strict Content-Security-Policy to reduce XSS blast radius.
 - **References:** OWASP A07:2021; CWE-922 Insecure Storage of Sensitive Information; CWE-79.
-

[MEDIUM] Apollo DevTools enabled in every environment

- **Category:** Security / GraphQL
- **Location:** `lib/hooks/useSetApollo.tsx:176`
- **Description:** `connectToDevTools: true` is hard-coded, so the Apollo Client DevTools bridge is active in production, exposing the full operation history, variables, and normalized cache (which includes the secret-laden configuration object and user PII).

- **Evidence:**

```
const client = new ApolloClient({ link: ..., cache, connectToDevTools: true });
```

- **Impact:** Anyone on the machine can inspect every query/mutation and cached sensitive data via the Apollo DevTools extension.
 - **Reproduction / How to verify:** Open the production build with the Apollo Client DevTools browser extension → cache & operations are fully visible.
 - **Recommended fix:** `connectToDevTools: process.env.NODE_ENV !== 'production'.`
 - **References:** OWASP A05:2021 Security Misconfiguration; CWE-489 Active Debug Code.
-

[MEDIUM] Sensitive values written to console

- **Category:** Security / Sensitive Data Exposure (logging)
- **Location:** `lib/context/global/google-maps.context.tsx:34; .../super-admin-layout/index.tsx:82,97; .../users/view/main/index.tsx:88` (88 `console.*` calls repo-wide)
- **Description:** Secrets and PII are logged to the browser console.
- **Evidence:**

```
// google-maps.context.tsx:34
console.log('GoogleMapsProvider - API Key:', apiKey);
// super-admin/index.tsx:82
console.log('▣ Push Token:', token);
// super-admin/index.tsx:97
console.log('▣ Foreground Notification:', payload);
// users/view/main/index.tsx:88
console.log('Row clicked:', event.data); // full user record (name, email,
phone, address)
```

- **Impact:** Google Maps key, FCM push token, notification payloads, and customer PII are exposed in console logs (captured by browser extensions, session-replay tools, error monitors).
 - **Reproduction / How to verify:** Open console on the dashboard / users page; observe the logged key, token, and PII.
 - **Recommended fix:** Remove these logs; add `no-console` (allow `warn/error`) to ESLint and strip via compiler in production builds.
 - **References:** OWASP A09:2021 Security Logging Failures; CWE-532 Insertion of Sensitive Information into Log File.
-

[MEDIUM] Next.js 14.2.5 — critical/high advisories

- **Category:** Security / Dependency
 - **Location:** `package.json` (`"next": "14.2.5"`, installed 14.2.5)
 - **Description:** `npm audit` flags `next` in the **critical** range (Cache Poisoning advisory, plus authorization-bypass/DoS advisories affecting 14.x).
 - **Evidence:** `npm audit → next: critical` via Next.js Cache Poisoning | range 0.9.9 - 16.3.0-canary.5.
 - **Impact:** Cache poisoning / potential middleware-authorization bypass depending on deployment.
 - **Reproduction / How to verify:** `npm audit | grep -A3 "next"`.
 - **Recommended fix:** Upgrade to the latest patched Next.js 14.2.x (or current 14 LTS line) and re-run `npm audit`; review the App-Router migration notes for the patch.
 - **References:** GHSA Next.js cache-poisoning advisories; OWASP A06:2021 Vulnerable & Outdated Components.
-

[MEDIUM] lodash _.template code-injection advisory (directly used)

- **Category:** Security / Dependency
 - **Location:** `package.json` (`"lodash": "^4.17.21"`); used via `import { throttle } from 'lodash'` (5 google-maps components) and `import { debounce } from 'lodash'` (`useQueryQL.tsx:9`, `useLazyQueryQL.tsx:8`).
 - **Description:** Audit flags `lodash <=4.17.23` as **high** (code injection via `_.template`). The app does not use `_.template`, but the vulnerable package is installed and shipped.
 - **Evidence:** `npm audit → lodash: high ... Code Injection via _.template | range <=4.17.23`.
 - **Impact:** No direct `_.template` call found, so practical exploitability here is low, but the flagged package is bundled.
 - **Reproduction / How to verify:** `npm ls lodash; grep -rn "_.template\|template(" lib app` (none found).
 - **Recommended fix:** Upgrade `lodash` to a patched release, or replace the only two helpers used (`throttle`, `debounce`) with `lodash/throttle` + `lodash/debounce` subpath imports / tiny local implementations and drop the full dependency.
 - **References:** GHSA lodash command/template injection; CWE-94.
-

[MEDIUM] next-intl open-redirect advisory

- **Category:** Security / Dependency
 - **Location:** `package.json` ("next-intl": "^3.26.3", installed 3.26.3)
 - **Description:** Audit flags next-intl <=4.9.1 for an **open redirect**. The app uses next-intl for locale routing (`i18n/request.ts`, `next.config.mjs`).
 - **Evidence:** `npm audit → next-intl: moderate ... open redirect | range <=4.9.1`.
 - **Impact:** Potential locale-prefixed redirect abuse depending on usage.
 - **Reproduction / How to verify:** `npm audit | grep next-intl`.
 - **Recommended fix:** Upgrade past 4.9.1 (note: a major bump from 3.x; plan the next-intl v4 migration) or apply the maintainer's patched 3.x if available.
 - **References:** GHSA next-intl open redirect; CWE-601.
-

[MEDIUM] Deprecated subscriptions-transport-ws used for the WebSocket link

- **Category:** Security / Dependency / GraphQL
- **Location:** `lib/hooks/useSetApollo.tsx:13,80-88`
- **Description:** The WS link uses the abandoned subscriptions-transport-ws SubscriptionClient, while the modern, maintained graphql-ws is declared in `package.json` but **never imported**. The deprecated library no longer receives security fixes.
- **Evidence:**

```
import { SubscriptionClient } from 'subscriptions-transport-ws';
const wsClient = new SubscriptionClient(`${WS_SERVER_URL}graphql`, {
  reconnect: true, timeout: 30000, lazy: true,
  connectionParams: () => ({ authorization: getAccessToken() ? `Bearer $
{getAccessToken()}` : '' }),
});
```

- **Impact:** Unmaintained transport in the auth-bearing realtime path; also a dependency duplication (two WS client libs).
- **Reproduction / How to verify:** `grep -rn "graphql-ws" lib app → no results;`
`grep -rn "subscriptions-transport-ws" lib app → only useSetApollo.tsx.`
- **Recommended fix:** Migrate to graphql-ws (`createClient` + `GraphQLWsLink`) and remove subscriptions-transport-ws.
- **References:** OWASP A06:2021.

[MEDIUM] CSRF posture / request-security notes (bearer model)

- **Category:** Security / CSRF
- **Location:** lib/hooks/useSetApollo.tsx:107-137 (request middleware), :75-88 (links)
- **Description:** Auth is conveyed as an `Authorization: Bearer ...` header read from `localStorage`, not via cookies. `createHttpLink` is created without `credentials`, so no ambient cookies are attached. This means **classic CSRF is not the primary risk** (the attacker cannot force the browser to attach the bearer header cross-site), but it also means there is no `SameSite/HttpOnly/Secure` protection on the credential (see the `localStorage` finding). No CSRF token mechanism is present, which is consistent with a header-bearer design.
- **Evidence:**

```
const httpLink = createHttpLink({ uri: `${SERVER_URL}graphql` }); // no
credentials: 'include'
operation.setContext({ headers: { authorization: token ? `Bearer ${token}` : '',
nonce, 'bop-auth': ... } });
```

- **Impact:** CSRF risk low for state-changing GraphQL (POST + custom header). CORS/cookie behavior of the API is a server concern — **Cannot be verified from the current codebase.**
- **Reproduction / How to verify:** Network tab → GraphQL POSTs carry authorization header, no Cookie/credentials.
- **Recommended fix:** Keep bearer-in-header (acceptable for CSRF), but pair with the cookie/CSP hardening from the `localStorage` finding; confirm backend CORS allow-list is restrictive.
- **References:** OWASP A01/A05:2021; CWE-352.

[LOW] bop-auth header built with a no-op || '' fallback

- **Category:** Security / GraphQL (correctness)
- **Location:** lib/hooks/useSetApollo.tsx:134
- **Description:** ``Bearer ${metricsToken}` || ''` — the template literal is always truthy, so the `|| ''` never triggers; when `metricsToken` is `null` the header becomes the literal string `"Bearer null"` instead of empty.
- **Evidence:**

```
'bop-auth': `Bearer ${metricsToken}` || '',
```


- **Impact:** Sends `Bearer null` for unauthenticated/public requests before the metrics token resolves; minor protocol noise, possible backend confusion.
 - **Reproduction / How to verify:** Clear local storage, trigger a public query before token fetch; inspect the `bop-auth` request header.
 - **Recommended fix:** `'bop-auth': metricsToken ? `Bearer ${metricsToken}` : ''`.
 - **References:** CWE-665 Improper Initialization.
-

[LOW] `next.config.mjs`: remote SVG allowed + unoptimized images + wildcard S3 host

- **Category:** Security / XSS / Misconfiguration
- **Location:** `next.config.mjs:5-9` and `:` (remotePatterns block, incl. `*.s3*.amazonaws.com`)
- **Description:** `dangerouslyAllowSVG: true` with `images.unoptimized: true` lets remote SVGs render; SVGs can embed scripts. The remote allow-list is broad (wildcard `*.s3*.amazonaws.com`, plus `example.com`, `freepick.com`, etc.).
- **Evidence:**

```
images: {
  unoptimized: true,
  dangerouslyAllowSVG: true,
  remotePatterns: [ ... { protocol: 'https', hostname: '*.s3*.amazonaws.com' } ],
}
```

- **Impact:** A malicious SVG served from an allowed remote host could execute script if rendered inline (mitigated when served via `<img>/Next Image`, which the codebase uses). Broad host allow-list weakens the image SSRF/abuse boundary. (Next sets `content-security-policy: script-src 'none'` for SVGs by default, lowering practical XSS risk.)
 - **Reproduction / How to verify:** Configure an entity image URL pointing to an attacker SVG on an allowed host; confirm how it renders.
 - **Recommended fix:** Set `dangerouslyAllowSVG: false` unless required; tighten `remotePatterns` to the actual CDN/S3 bucket; remove placeholder hosts (`example.com`, `freepick.com`).
 - **References:** OWASP A05:2021; CWE-79.
-

[INFO] Hardcoded Firebase web config in the public service worker

- **Category:** Security / Secrets (informational)

- **Location:** `public/firebase-messaging-sw.js:9-17`
- **Description:** The FCM service worker hardcodes a Firebase config (`apiKey: 'AIzaSyD...SnJ3U'`, `projectId: 'enatega-multivender-web'`). Firebase **web** API keys are public by design (they identify, not authorize, the project), so this is not a secret leak per se — but the admin SW points at the **customer-web** Firebase project, which looks like a copy-paste mismatch, and the notification icon points at a random `pngegg.com` image.
- **Evidence:**

```
firebase.initializeApp({ apiKey: 'AIzaSyDx_iSQ9LroTF7NMm20aRvw2wJqhwSnJ3U',
  authDomain: 'enatega-multivender-web.firebaseio.com', projectId: 'enatega-multivender-web', ... });
```

- **Impact:** Informational; cross-project notification routing bug rather than a credential leak.
 - **Reproduction / How to verify:** Open `public/firebase-messaging-sw.js`; compare project to the admin's intended FCM project (config elsewhere comes from `ConfigurationContext`).
 - **Recommended fix:** Provision SW config from build-time env for the correct project; restrict the Firebase web key by HTTP referrer in the Google Cloud console; replace the placeholder icon.
 - **References:** Firebase docs (API keys are public); CWE-1188.
-

[LOW] External redirects via `window.open` / `window.location.href`

- **Category:** Security / Open Redirect
- **Location:** `.../super-admin-layout/index.tsx:106`; `.../restaurant/payment/main/index.tsx:48`; `.../side-bar/side-bar-item.tsx:95,100`
- **Description:** A hardcoded foreign dashboard URL is opened on notification click, and a payment screen navigates to a backend-supplied `data.url`.
- **Evidence:**

```
// super-admin/index.tsx:106
notification.onclick = () => { window.open('https://multivendor-admin.ninjascode.com/dashboard'); };
// restaurant/payment/main/index.tsx:48
window.location.href = data.url; // backend-provided
```

- **Impact:** The `ninjascode.com` redirect is a legacy/wrong destination (also a branding bug). `data.url` is backend-controlled (Stripe Connect onboarding, typically) — low risk if the API is trusted; **Cannot be verified from the current codebase** that `data.url` is constrained server-side.

- **Reproduction / How to verify:** Trigger a foreground push and click it → lands on `ninjascode.com`. Inspect the `payment.data.url` source.
- **Recommended fix:** Replace the hardcoded URL with the app's own route; validate/allow-list any redirect target derived from server data.
- **References:** CWE-601 Open Redirect.

Authentication & Session — checklist summary (verified against code)

Item	Finding	Evidence
Login implementation	Formik + Yup → <code>OWNER_LOGIN</code> mutation	<code>sign-in-email-password/index.tsx:66-100</code>
Token storage location	localStorage (token) + full payload (user - ...)	<code>auth.ts:15</code> , <code>login:73</code>
JWT expiry handling	<code>tokenExpiration</code> stored; metrics token has refresh logic; app JWT has no auto-logout	<code>auth.ts:18</code> , <code>security.ts:57-82</code>
Logout behavior	Does not clear token/userType/metrics token	<code>app-bar:163-170</code>
Auto logout on expiry	Not implemented for the app JWT	no call site found
Refresh-token handling	Only for the public "metrics/bop-auth" token, not the user JWT	<code>security.ts</code> , <code>useSetApollo.tsx:30-67</code>
Hardcoded tokens	None in app code; Firebase web key in SW (public by design)	<code>firebase-messaging-sw.js:10</code>
Remember-me	Not present (session persists via <code>localStorage</code> until manual logout)	—
Unauth access to protected pages	Guard race allows brief render + queries	<code>*_GUARD.tsx</code>
Protected routes enforced server-side?	Cannot be verified from the current codebase	backend not in repo

PART 3 — Performance Findings

[HIGH] Fetch-all-rows then filter & paginate client-side

- **Category:** Performance / GraphQL / React Render
- **Location:**
lib/ui/screen-components/protected/super-admin/users/view/main/index.tsx:39-82 (representative; same shape across many .../view/main/index.tsx screens)
- **Description:** GET_USERS fetches **every** user with PII (name, email, phone, addresses, coordinates) and **no pagination/filter arguments**; the component then filters with `Array.filter` and paginates with `Array.slice` in the browser. Search, registration-method and status filters are all client-side.
- **Evidence:**

```
const { data } = useQuery<IUsersDataResponse>(GET_USERS, { fetchPolicy:
'network-only' });
const allUsers = useMemo(() => data?.users ?? [], [data]);
const filteredUsers = useMemo(() => { /* .filter by search/type/status */ },
[...]);
const paginatedUsers = useMemo(() => filteredUsers.slice(start, end), [...]);

# queries/user/index.ts
query users { users { _id name email phone createdAt userType status lastLogin
notes addresses { ... } } }
```

- **Impact:** Transfer + parse + memory scale with the **entire** table; on a real deployment with tens of thousands of users this means multi-MB responses, long main-thread JSON parsing, and a sluggish page — while only 10 rows are shown. Also over-fetches PII to the client.
- **Reproduction / How to verify:** Seed many users → open `/general/users` → Network shows one large `users` response; Performance shows a long parse/render; pagination/search never hit the network.
- **Recommended fix:** Add server-side pagination/search/filter arguments (`limit`, `skip/cursor`, `search`, `status`) and request only the columns the table renders; bind PrimeReact's `lazy pagination/onPage` to refetch.
- **References:** OWASP API ("Excessive Data Exposure" also a perf cost); Apollo pagination docs.

[HIGH] fetchPolicy: 'network-only' used 55× (cache effectively disabled)

- **Category:** Performance / GraphQL
- **Location:** repo-wide (55 occurrences; e.g. `users/view/main/index.tsx:40`), vs 16 `cache-and-network`, 1 `no-cache`, 1 `cache-first`
- **Description:** The dominant fetch policy bypasses the Apollo cache, so navigating back to a

screen always re-hits the network even when data is unchanged.

- **Evidence:** `grep -c "fetchPolicy: 'network-only'"` → 55.
- **Impact:** Redundant round-trips, slower navigation, more backend load; the normalized cache provides little benefit.
- **Reproduction / How to verify:** Navigate away and back to a list screen → a fresh network request fires each time.
- **Recommended fix:** Default to `cache-first` / `cache-and-network` for list/detail reads; reserve `network-only` for truly volatile data (live dispatch). Centralize per-operation policy in the `useQueryGQL` wrapper.
- **References:** Apollo Client caching guide.

[MEDIUM] 654-line dummy-data generator shipped to production as loading skeletons

- **Category:** Performance / Bundle / Cleanup
- **Location:** `lib/utils/dummy/index.tsx` (654 lines, ~24 `generate*` exports); imported by 24 `view/main` screens, e.g. `users/view/main/index.tsx:5, 94`
- **Description:** Screens render fabricated rows while loading: `data={loading ? generateDummyUsers() : paginatedUsers}`. The full generator module (fake users, orders, riders, restaurants, etc.) is bundled into the client.
- **Evidence:**

```
import { generateDummyUsers } from '@lib/utils/dummy';  
<Table data={loading ? generateDummyUsers() : paginatedUsers} ... />
```

- **Impact:** Extra JS in the bundle on every route; fake data momentarily visible; maintenance noise.
- **Reproduction / How to verify:** `grep -rln "utils/dummy"` → 24 files; `build + bundle-analyzer` shows the module included.
- **Recommended fix:** Replace dummy-data placeholders with the existing skeleton components (`lib/ui/useable-components/custom-skeletons/*`), then delete `lib/utils/dummy`.
- **Removal safety: Medium risk** — used by 24 screens; migrate each to skeletons before deleting.

[MEDIUM] moment bundled for a single usage while date-fns is unused

- **Category:** Performance / Bundle / Duplicate libraries
- **Location:** `package.json` (moment, date-fns both present); moment imported only in `.../audit-logs/AuditLogCard.tsx`; date-fns imported **nowhere**

- **Description:** Two date libraries are declared. `moment` (large, non-tree-shakeable) is used in exactly one component; `date-fns` (modern, tree-shakeable) has zero imports.
- **Evidence:** `grep -rln "from 'moment'" → 1 file`; `grep -rn "date-fns" lib app → 0`.
- **Impact:** Unnecessary bundle weight from `moment`; a dead `date-fns` dependency.
- **Reproduction / How to verify:** the greps above; `bundle-analyzer`.
- **Recommended fix:** Rewrite the one `moment` call with `Intl.DateTimeFormat` or `date-fns`, remove `moment`; if `date-fns` is not adopted, remove it too.
- **Removal safety:** **Medium risk** (verify the single audit-log formatting).

[MEDIUM] Global `images.unoptimized: true`

- **Category:** Performance / Bundle / Images
- **Location:** `next.config.mjs:7`
- **Description:** Next.js image optimization (resizing, modern formats, lazy sizing) is disabled platform-wide, so original images are served as-is.
- **Evidence:** `images: { unoptimized: true, dangerouslyAllowSVG: true, ... }`.
- **Impact:** Larger image payloads and slower LCP on dashboards with restaurant/banner imagery.
- **Reproduction / How to verify:** Network tab on a banners/restaurants page → full-size originals; no `/_next/image` optimization.
- **Recommended fix:** Enable optimization (remove `unoptimized: true`) and rely on the configured `remotePatterns`; if a static export forces it, document the constraint.

[LOW] Live monitor polls every 15s with network-only

- **Category:** Performance / GraphQL
- **Location:** `lib/ui/screen-components/protected/vendor/dashboard/live-monitor/index.tsx:36`
- **Description:** `pollInterval: 15000` continuously refetches while the dashboard is open.
- **Evidence:** `pollInterval: 15000,.`
- **Impact:** Steady background query load per open dashboard; multiplied across vendors.
- **Reproduction / How to verify:** Open the vendor dashboard → Network shows a query every 15s indefinitely.
- **Recommended fix:** Prefer a GraphQL subscription (WS already configured) for live orders,

or pause polling when the tab is hidden (`document.visibilityState`) and back off when idle.

[LOW] `React.memo` virtually unused on heavy list/cell components

- **Category:** Performance / React Render
 - **Location:** repo-wide (only 2 `React.memo` across 742 files; `useMemo` 95×, `useCallback` 43×, `useEffect` 103×)
 - **Description:** Column renderers, action menus (e.g. `users/view/main/ActionMenu.tsx`) and table cells re-render with parent state (open-menu id, pagination) without memoization. Note `useMemo/useCallback` are used reasonably in many places, so this is targeted, not systemic.
 - **Evidence:** `grep -c "React.memo" lib app → 2.`
 - **Impact:** Avoidable re-renders on large tables, especially with per-row menus.
 - **Reproduction / How to verify:** React Profiler while paging/opening row menus on `/general/users`.
 - **Recommended fix:** Wrap pure cell/row/menu components in `React.memo`; pass stable callbacks (already `useCallback`'d) and avoid inline object props in column defs.
-

PART 4 — Cleanup / Dead-Code / Irrelevant-Code Findings

[MEDIUM] `clearAuthTokens` / `clearMetricsData` defined but never called

- **Category:** Cleanup / Dead Code (with a security consequence)
- **Location:** `lib/utils/methods/auth.ts:32;`
`lib/utils/methods/security.ts:84`
- **Why it appears unused:** `grep -rn "clearAuthTokens\|clearMetricsData" app lib` returns only the definitions — no call sites. This is the root cause of the HIGH logout finding.
- **How to verify safely:** Confirm no references before changing; then **wire them into logout** rather than deleting (they are the correct cleanup functions).
- **Removal safety: Medium risk** — do **not** delete; integrate into `onConfirmLogout` across the three app-bars.

[MEDIUM] 25 orphaned locale files (only 6 locales are wired up)

- **Category:** Cleanup / Unused translations
- **Location:** `locales/*.json` (31 files) vs `config.ts` (6 locales)
- **Why it appears unused:** `config.ts` declares `['ar', 'de', 'en', 'fr', 'km', 'zh'];` `i18n/request.ts` imports `../locales/${locale}.json` where `locale` comes from `getUserLocale()`, and only the 6 are selectable. The other 25 (`az, bn, es, fa, gu, he, hi, id, it, jp, ko, ku, mr, nl, pl, ps, pt, ro, ru, te, th, tr, ur, uz, vi`) are never reachable.
- **How to verify safely:** Confirm the locale switcher only exposes the 6 (`config.ts` + locale dropdown); `grep -rn "locales/" lib app i18n`.
- **Removal safety: Medium risk** — verify there is no plan to enable them; they're harmless but inflate the repo and translation workload.

[MEDIUM] Legacy white-label branding hardcoded

- **Category:** Cleanup / Irrelevant (legacy) code
- **Location:** `.../super-admin-layout/side-bar/side-bar-item.tsx:97-98;`
`.../super-admin-layout/index.tsx:106`
- **Why it appears irrelevant:** `hedgego.com.au / hedge.net.au` and `multivendor-admin.ninjascode.com` are not Enatega domains — leftovers from a white-label/previous deployment.
- **Evidence:**

```
route === 'https://hedgego.com.au/' || route === 'https://hedge.net.au/become-a-stockist'
...
window.open('https://multivendor-admin.ninjascode.com/dashboard');
```

- **How to verify safely:** `grep -rn "hedge\\|ninjascode" lib app`; confirm with product owner these aren't intended partner links.
- **Removal safety: Medium risk** — confirm before removing (sidebar may intentionally link a partner); the `ninjascode` notification target is almost certainly a bug.

[LOW] Default Cypress scaffold instead of real tests

- **Category:** Cleanup / Dead test code
- **Location:** `cypress/e2e/1-getting-started/todo.cy.ts,`
`cypress/fixtures/example.json`
- **Why it appears unused:** These are the files `cypress open` generates; no app-specific specs exist.
- **How to verify safely:** Inspect the spec (it's the generic `TodoMVC` sample).

- **Removal safety:** **Safe** to delete the scaffold (ideally replace with real auth/RBAC/table specs from the QA plan below).

[LOW] graphql-ws declared but unused (duplicate WS client)

- **Category:** Cleanup / Duplicate dependency
- **Location:** `package.json ("graphql-ws": "^5.16.0")`
- **Why it appears unused:** `grep -rn "graphql-ws" lib app → 0`; the app uses `subscriptions-transport-ws`.
- **How to verify safely:** the `grep` above + `npm ls graphql-ws`.
- **Removal safety:** **Safe** to remove — **or** preferably migrate the WS link to `graphql-ws` and remove `subscriptions-transport-ws` instead (see MEDIUM security finding).

[LOW] Debug console.* statements throughout (88)

- **Category:** Cleanup / Debug code
- **Location:** repo-wide (88 calls); notable: `google-maps.context.tsx:34`, `super-admin/index.tsx:82, 90, 97`, `users/view/main/index.tsx:88`
- **Why it appears unused:** Development logging left in production code; several leak secrets/PII (see MEDIUM security finding).
- **How to verify safely:** `grep -rn "console\." lib app`.
- **Removal safety:** **Safe** — remove or guard behind a logger; enforce with ESLint `no-console`.

[LOW] Commented-out code blocks / dead branches

- **Category:** Cleanup / Commented code
- **Location:** e.g. `lib/hoc/RESTAURANT_GUARD.tsx:37-39` (commented `VENDOR` redirect); broad comment volume across the tree (27 `eslint-disable`, 1 `TODO`)
- **Why it appears unused:** Commented logic such as the disabled `VENDOR` check leaves the guard's intent ambiguous.
- **Evidence:**

```
// RESTAURANT_GUARD.tsx:37
// if (user?.userType === 'VENDOR') { router.replace('/forbidden'); }
```

- **How to verify safely:** Review each block against current product rules; the `RESTAURANT_GUARD` comment in particular changes who can access restaurant routes.
- **Removal safety:** **Medium risk** for the guard comment (it's a real access-control decision); **Safe** for pure dead comments.

[INFO] GET_USERS_L lightweight query — confirm intent

- **Category:** Cleanup / Possibly redundant query
 - **Location:** lib/api/graphql/queries/user/index.ts
 - **Why it appears notable:** GET_USERS_L returns only `_id` (likely a count helper) alongside the heavy GET_USERS.
 - **How to verify safely:** `grep -rn "GET_USERS_L" lib app` to confirm it is referenced; if unused, remove.
 - **Removal safety:** Needs developer confirmation.
-

PART 5 — Prioritized Remediation Roadmap

Immediate Fixes (security — do first)

1. **Clear all auth state on logout** — call `clearAuthTokens()` + `clearMetricsData()` + `apolloClient.clearStore()` in all three `onConfirmLogout` handlers. (*HIGH*)
2. **Stop fetching secrets to the client** — remove `secretKey`, `clientSecret`, `password`, `twilioAuthToken`, `sendGridApiKey`, `sendGridPassword` from `GET_CONFIGURATION`; coordinate a write-only secret pattern with the backend. (*CRITICAL*)
3. **Fix the guards** — add `user` to `deps`, gate rendering with a loader, return `null` while unauthorized. (*HIGH*)
4. **Disable Apollo DevTools in production** (`connectToDevTools: NODE_ENV !== 'production'`). (*MEDIUM*)
5. **Remove secret/PII console.* logs**; add ESLint `no-console`. (*MEDIUM*)
6. **Confirm backend authorization** on every query/mutation independent of the client role. (*HIGH — backend*)

Short-Term Fixes (performance & maintainability)

1. Add server-side pagination/search/filter to the heavy list queries (`users`, `orders`, `restaurants`, `riders`, ...); stop client-side `slice/filter`. (*HIGH perf*)
2. Re-tune fetch policies away from blanket `network-only`. (*HIGH perf*)
3. Replace `lib/utils/dummy` placeholders with skeleton components and delete the module. (*MEDIUM*)
4. Remove `moment` (rewrite the one usage); drop unused `date-fns`. (*MEDIUM*)
5. Enable Next image optimization; tighten `remotePatterns`; disable remote SVG. (*MEDIUM*)

Larger Refactors (structural)

1. Move the session token to an `HttpOnly/Secure/SameSite` cookie + add a strict CSP.
(*security architecture*)
2. Migrate WS to `graphql-ws`; remove `subscriptions-transport-ws`.
(*dependency*)
3. Plan dependency upgrades (Next.js, next-intl v4, lodash) behind a regression pass.
(*dependency*)
4. Convert aggressive polling (live monitor) to subscriptions with visibility-based pausing.
(*perf*)

Safe Cleanup Checklist

- ☐ Remove default Cypress scaffold:
`cypress/e2e/1-getting-started/todo.cy.ts`,
`cypress/fixtures/example.json`
- ☐ Remove unused dependency: `graphql-ws` (or migrate WS to it and drop `subscriptions-transport-ws`)
- ☐ Remove unused dependency: `date-fns` (if not adopted)
- ☐ Remove debug/PII logs from: `lib/context/global/google-maps.context.tsx:34`, `.../super-admin-layout/index.tsx:82, 90, 97`, `.../users/view/main/index.tsx:88`
- ☐ Remove legacy branding redirect: `.../super-admin-layout/index.tsx:106` (`ninjascode.com`)
- ☐ Confirm before deleting: `lib/utils/dummy/index.tsx` (migrate 24 screens to skeletons first)
- ☐ Confirm before deleting: 25 orphaned `locales/*.json`
- ☐ Confirm before deleting: sidebar `hedgego.com.au / hedge.net.au` links (`sidebar-item.tsx:97-98`)
- ☐ Confirm before deleting: `GET_USERS_L` query
- ☐ Do **not** delete `clearAuthTokens/clearMetricsData` — wire them into logout

Developer Implementation Checklist

- **Security**
 - ☐ Wire `clearAuthTokens()` + `clearMetricsData()` + `client.clearStore()` into all logout handlers
 - ☐ Remove secret fields from `GET_CONFIGURATION`; make config-form secrets write-only
 - ☐ Fix guard deps/render-gating in

SUPER_ADMIN_GUARD/VENDOR_GUARD/RESTAURANT_GUARD

- ☐ connectToDevTools gated by env
- ☐ Fix bop-auth header || '' logic (useSetApollo.tsx:134)
- ☐ dangerouslyAllowSVG: false; tighten remotePatterns
- ☐ Verify/repair Firebase SW project + key referrer restriction
- **Performance**
 - ☐ Server-side pagination/filter for list queries; request only displayed fields
 - ☐ Audit & reduce network-only usage
 - ☐ Skeletons instead of dummy data; remove lib/utils/dummy
 - ☐ Drop moment; enable image optimization
 - ☐ React.memo on hot row/cell/menu components
- **Dependency upgrades**
 - ☐ next → latest patched 14.2.x; next-intl → patched/major; lodash → patched (or subpath imports); migrate WS client
 - ☐ Re-run npm audit to confirm critical/high count drops
- **Testing/QA**
 - ☐ Implement the QA Test Plan below as Cypress specs

QA Test Plan Checklist

- ☐ **Login (valid)**
 - Page/Component: /authentication/login, sign-in-email-password/index.tsx
 - Steps: enter valid admin creds → submit
 - Expected result: redirect to role default route; success toast
 - Evidence to collect: network OWNER_LOGIN 200; localStorage token/user -... set
- ☐ **Login (invalid)**
 - Steps: wrong password → submit
 - Expected: error toast; no token written
 - Evidence: error toast screenshot; localStorage empty
- ☐ **Logout clears session** (*regression for HIGH finding*)
 - Page/Component: super-admin app-bar
 - Steps: login → logout → inspect localStorage → trigger a query
 - Expected: token, userType, userId, tokenExpiration, metrics token all removed; no Bearer header on subsequent requests
 - Evidence: before/after localStorage; Network headers

- ☐ **Expired token**
 - Steps: set `tokenExpiration` to the past → make a request
 - Expected: user is forced to re-authenticate (not silently allowed)
 - Evidence: redirect to login / 401 handling
- ☐ **Protected route while unauthenticated**
 - Steps: clear storage → visit `/general/users`
 - Expected: redirect to login with no data flash
 - Evidence: screen recording (no content/query before redirect)
- ☐ **Role manipulation via localStorage** (*privilege escalation*)
 - Steps: login as vendor/staff → edit user -... `userType` to ADMIN/SUPER ADMIN → reload → attempt a super-admin mutation
 - Expected: backend rejects the mutation (server-side authz)
 - Evidence: GraphQL response (must be authorization error)
- ☐ **XSS payloads in entity fields**
 - Page/Component: vendor/product/customer/order create+edit forms, tables, audit-log diff, notifications
 - Steps: submit `<img src=x onerror=alert(1)>`, `"><script>alert(1)</script>, javascript:alert(1)` in name/notes/address fields → view in tables/modals/audit log
 - Expected: rendered as inert text (React escaping), no script execution
 - Evidence: no alert; DOM shows escaped text
- ☐ **Unauthorized admin mutation**
 - Steps: as vendor, call a super-admin-only mutation (e.g. `DELETE_USER`) via the GraphQL client
 - Expected: server denies
 - Evidence: response error
- ☐ **Bulk/destructive delete confirmation**
 - Page/Component: `delete-dialog`, entity action menus
 - Steps: trigger delete on a restaurant/user/coupon
 - Expected: confirmation dialog appears before mutation fires
 - Evidence: dialog screenshot; mutation only after confirm
- ☐ **Large table performance**
 - Page/Component: `/general/users`
 - Steps: seed many rows → load page → measure
 - Expected (post-fix): only a page of rows fetched; fast render

- Evidence: Network payload size; Performance profile
- ☐ **Search debounce**
 - Steps: type quickly in users search
 - Expected (post-fix): debounced server query, not one per keystroke
 - Evidence: Network request count
- ☐ **Pagination**
 - Steps: page through users/orders
 - Expected (post-fix): each page change triggers a bounded server fetch
 - Evidence: Network per page
- ☐ **Duplicate API call check**
 - Steps: navigate away and back to a list screen
 - Expected (post-fix): served from cache where appropriate, not always network-only
 - Evidence: Network tab
- ☐ **Removed-code regression**
 - Steps: after deleting dummy/locales/scaffold, run full smoke (all screens load, loading states render)
 - Expected: no missing-import/runtime errors
 - Evidence: console clean; screens render skeletons

PART 6 — Dependency Upgrade Plan

npm audit summary: **30 vulnerabilities** — 3 critical, 14 high, 12 moderate, 1 low.

Package	Installed	Severity	Issue (advisory)	Direct use in code?	Recommended action
next	14.2.5	Critical	Cache poisoning (+ authz/DoS advisories)	Yes (framework)	Upgrade to latest patched 14.2.x; review App-Router notes
protobufjs	transitive	Critical	Arbitrary code execution	No (via firebase/grpc)	Upgrade firebase/grpc chain
form-data	transitive	Critical	Unsafe random boundary	No (via cypress/request)	Upgrade cypress
lodash	4.17.21	High	Code injection via	Yes (throttle, d	Patch upgrade or switch to

Package	Installed	Severity	Issue (advisory)	Direct use in code?	Recommended action
			_.template	ebounce)	subpath imports / drop
@grpc/grpc-js	transitive	High	Malformed request crash	No (firebase)	Upgrade firebase
next-intl	3.26.3	Moderate	Open redirect	Yes (i18n)	Upgrade past 4.9.1 (plan v4 migration)
ws	transitive	High	Memory-exhaustion DoS	No (WS stack)	Upgrade WS chain (migrate to graphql-ws)
@typescript-eslint/*	transitive (eslint)	High	minimatch/estree ReDoS	Dev only	Upgrade ESLint TS plugins
eslint-config-next / @next/eslint-plugin-next	14.2.5	High	glob CLI injection	Dev only	Upgrade with Next
cypress	13.x	Moderate	via @cypress/request/qs	Dev only	Upgrade Cypress
postcss	transitive	Moderate	XSS in stringify	Build only	Upgrade
@babel/runtime, ajv, brace-expansion, js-yaml, yaml, uuid, qs, fast-uri, flattened, picomatch, tmp, diff, minimatch	transitive	High/Moderate/Low	various ReDoS/traversal/pollution	Mostly dev/build	Resolve via top-level upgrades + npm audit fix

Duplicate/unused dependencies to resolve alongside: graphql-ws (unused) vs subscriptions-transport-ws (used, deprecated); moment (1 use) vs date-fns (unused).

Run `npm audit fix` for safe transitive bumps first, then schedule the framework majors (next, next-intl) behind the QA suite. Re-run `npm audit` and attach the

new counts to the PR.

PART 7 — Final Risk Rating

Overall: HIGH

Justification (verified evidence only):

- **Session management is broken client-side:** logout leaves a valid bearer token in `localStorage` (`app-bar/index.tsx:163`; `clearAuthTokens/clearMetricsData` never invoked), and the user JWT has no auto-logout.
- **Authorization is client-trusted and non-blocking:** role/permissions come from mutable `localStorage` (`user-context.tsx:17`) and guards neither re-run on user load nor gate rendering (`*_GUARD.tsx`). Real protection depends entirely on a backend that is **not in this repo and cannot be verified**.
- **Secret exposure:** the configuration query pulls Stripe/PayPal/Twilio/SendGrid/SMTP secrets into the browser (`queries/configuration/index.ts`).
- **Dependency surface:** 3 critical + 14 high advisories, including the Next.js critical line that is the running framework.

These are concrete, code-evidenced issues with high business impact (financial-secret exposure, session reuse, potential privilege escalation). XSS exposure is comparatively **low** (no `dangerouslySetInnerHTML/eval`, React escaping, Formik+Yup, confirmation dialogs on destructive actions), which prevents a *Critical* overall rating — but it would rise to Critical if the backend is found to rely on the client-side role checks.

Rating would drop to MEDIUM once: logout fully clears session state, guards gate rendering, secrets are removed from the client config query, Apollo DevTools is disabled in prod, and the critical/high dependencies are upgraded — **and** backend per-operation authorization is confirmed.

End of report. All findings are tied to code present in `enatega-multivendor-admin` on branch `adeel` as of 2026-06-29. Items dependent on the proprietary backend are explicitly labeled and were not assumed.